

网络流的精髓在于建图。建图是人类智慧。

网络流本质上是贪心。网络流的建图刻画了问题本身的 **可贪心** 性质，从而简单地将问题的反悔操作统一为图上的网络流算法的反悔操作，使得我们不需要为每个问题都寻找对应的反悔策略。

1. 基本定义

一个网络流是一张 **有向图** $G = (V, E)$ 。每条有向边 $(x, y) \in E$ 有 **容量** $c(x, y)$ 和 **流量** $f(x, y)$ 。特别地，若 $(x, y) \notin E$ ，则 $c(x, y) = 0$ 。 f 称为网络流的 **流函数**。

注意

流函数对所有二元有序对均有定义，而不仅仅只是图上的边。

网络流的 **可行流** 分为有源汇和无源汇两种情况，常用 S 表示源点， T 表示汇点。一般认为源点没有入度，汇点没有出度。

可行流的流函数 f 满足以下性质：

- **容量限制**：流量不超过容量。若 $f(x, y) = c(x, y)$ ，则称 $x \rightarrow y$ **满流**。
- **斜对称**： $f(x, y) = -f(y, x)$ 。 $x \rightarrow y$ 有 1 的流量，也可以认为 $y \rightarrow x$ 有 -1 的流量。
- **流量守恒**：除源汇点以外（无源汇网络流不存在源汇点），从每个点流入和流出的流量相等。每个点不储存流量，流进多少就流出多少。

对于有源汇网络流，根据斜对称和容量守恒性质，从源点流出的总流量等于流入汇点的总流量。这个相等的值称为 f 的 **流量**。

f 在 G 上的 **残量网络** $G_f = (V, E_f)$ 是容量函数等于 $c_f = c - f$ 的网络。根据容量限制， $c_f(x, y) \geq 0$ 。若 $c_f(x, y) = 0$ ，则 $x \rightarrow y$ 在残量网络上不存在。

容量限制等价于残量网络的容量非负。

注意

残量网络上可以有原图没有的边 $y \rightarrow x$ ，此时其对应的反边 $x \rightarrow y$ 有正流量，所以 $y \rightarrow x$ 有负流量， $c_f(y, x) > 0$ 。

- **增广路**：残量网络 G_f 上从源点到汇点的路径称为 **增广路**。无源汇网络流不讨论增广路。
- **割**：将 V 划分为 **互不相交** 的两个点集 A, B ，其中 $S \in A, T \in B$ ，称为 **割** (A, B) 。定义割的 **容量** 为从 A 指向 B 的所有边的容量之和。若 x, y 所属点集不同，则称 $x \rightarrow y$ 为 **割边**。

接下来讨论的内容默认图有源汇点。关于无源汇网络流，见 4.1 小节。

注

虽然我们是对所有结点有序对定义的流量，但也可以对每条边定义流量，以及其对应残量网络上的反边。每条边（以及它们的流量）是相互独立的，每条边在残量网络上的反边也是相互独立的，且和原网络上的边独立：

- 当图有重边时，所有重边之间互不影响，可以看成独立的若干条边，它们也有对应独立的反边。
- 当图有反边时，一条边和它的反边互不影响。原图上的反边的流量是反边的真实流量，而残量网络上的反边的流量表示原图上这条边还能流多少流量。

这使得我们不需要考虑重边和反边的情况，给相关算法的实现带来了很大的方便。

2. 网络最大流

给定网络 $G = (V, E)$ 和源汇，求可行流的最大流量 *maximum flow*, MF 。

本文只介绍最经典的 Edmonds-Karp 和 Dinic，不介绍 SAP，ISAP 和 HLPP。

2.1 增广

接下来的两个算法均使用了 **不断寻找增广路** 和 **能流满就流满** 的贪心思想。

具体地，找到残量网络 G_f 上的一条增广路 P ，并为 P 上的每条边增加 $c_f(P) = \min_{(x,y) \in P} c_f(x,y)$ 的流量。如果增加的流量大于该值，一些边将不满足容量限制。而根据能流满就流满的思想，增加的流量也不应小于该值。

如果 $f(x,y)$ 加上了 C ，那么根据斜对称性质， $f(y,x)$ 减去了 C 。对应地，在残量网络上， $c_f(x,y)$ 减去 C ， $c_f(y,x)$ 加上 C 。因为我们操作的对象是残量网络，所以相当于将 $x \rightarrow y$ 的容量减去 C ，然后将 $y \rightarrow x$ 的容量加上 C 。

上述操作称为一次 **增广**，可以理解为：

- 容量限制等价于残量网络的容量非负。在残量网络上，我们只关心每条边的剩余容量。
- 将一条增广路上每条边的流量加 c ，等价于将每条边在残量网络上的容量减 c 。
- 将一条边的流量加 c ，根据斜对称性质，需要将反边的流量减 c ，等价于将反边在残量网络上的容量加 c 。
- 因为我们是将一条路径上的每条边的流量加 c ，所以自然满足流量守恒性质。

- 从实际意义的角度理解这个过程，就是增加当前边的流量时需要增加反边在残量网络上的容量，这样流过反边的时候可以收回当前边的一部分流量，支持“反悔”。

常用技巧：**成对变换**。网络流建图一般使用链式前向星。我们将每条边与它的反向边按编号连续存储，分别编号为 $2k$ 和 $2k + 1$ ，从而快速求得编号为 x 的边的反向边的编号为 $x \oplus 1$ 。为此，初始 `cnt` 应设为 1！

2.2 最大流最小割定理

最大流最小割定理：网络的最大流量等于割的最小容量。

最大流最小割定理是贯穿网络流始终的核心。它是网络流相关理论的最基本结论。

结论

任意可行流的流量不大于任意割的容量。

证明

考虑任何可行流 f 和任意割 (A, B) 。每单位流量至少一次经过从 A 到 B 的割边，所以

$$|f| \leq \sum_{(x,y) \in (A,B)} f(x,y) \leq \sum_{(x,y) \in (A,B)} c(x,y) = c(A,B).$$

□

将流的大小理解为单位时间可以流过多少体积的水（不可压缩液体）。对任何割，单位时间内从一侧流到另一侧的水的体积（流的流量）不超过割的容量。

结论

对于整数容量网络，存在一个流的流量等于一个割的容量。

证明

S 到 T 的流量有上界，所以最大流量存在。因为网络是整数容量的，所以每次增广至少增加 1 流量。

在有限次增广之后，在残量网络上， S 和 T 不连通，否则可以继续增广。设 A 为残量网络上 S 可达的所有点， B 为剩下的所有点。

因为 A 在残量网络上不可达 B ，所以不可能有从 B 到 A 的边有流量（第一个等号），且 A 到 B 的所有割边满流（第二个等号）。因此，每单位流量恰好一次经过从 A 到 B 的割边，

$$|f| = \sum_{(x,y) \in (A,B)} f(x,y) = \sum_{(x,y) \in (A,B)} c(x,y) = c(A,B).$$

□

如果第一个集合的任意一个数不大于第二个集合的任意一个数，且存在第一个集合的一个数和第二个集合的一个数相等，那么第一个集合的最大值等于第二个集合的最小值。因此，最大流等于最小割。□

最大流最小割定理告诉我们：**如果在一个可行流的残量网络上 S 和 T 不连通，那么这个可行流是最大流。**

最大流最小割定理也给出了整数容量网络的最大流算法：在残量网络上不断增广，直到残量网络不连通。但这个朴素的算法（Ford-Fulkerson, FF）的时间复杂度和流量有关，较劣。

2.3 Edmonds-Karp 算法

算法介绍

任意选择增广路不优秀，那就选择具有一定性质的增广路。

Edmonds-Karp 算法的核心是使用 BFS 寻找 **长度最短** 的增广路。为此，对当前增广路上的每个点，我们记录流向这个点的边的编号（方便修改容量），然后从汇点 T 不断回推到源点 S 。时间复杂度 $\mathcal{O}(nm^2)$ 。

[模板题](#) 代码如下。

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 200 + 5, M = 5e3 + 5;
struct flow {
    ll fl[N], limit[M << 1];
    int cnt = 1, hd[N], nxt[M << 1], to[M << 1], fr[N];
    void add(int u, int v, int w) {
        nxt[++cnt] = hd[u], hd[u] = cnt, to[cnt] = v, limit[cnt] = w;
        nxt[++cnt] = hd[v], hd[v] = cnt, to[cnt] = u, limit[cnt] = 0;
    }
    ll maxflow(int s, int t) {
        ll flow = 0;
        while(1) {
            queue<int> q;
            memset(fl, -1, sizeof(fl));
            fl[s] = 1e18, q.push(s);
            while(!q.empty()) {
                int t = q.front();
                q.pop();
                for(int i = hd[t]; i; i = nxt[i]) {
                    int it = to[i];
                    if(limit[i] && fl[it] == -1) { // 若剩余流量为 0, 那么在残量网络上不存在, 不能走
                        fl[it] = min(limit[i], fl[t]); // 记录流量
                        fr[it] = i, q.push(it); // 记录流向每个点的边的编号
                    }
                }
            }
            if(fl[t] == -1) return flow;
            flow += fl[t];
            for(int u = t; u != s; u = to[fr[u] ^ 1]) {
                limit[fr[u]] -= fl[t];
                limit[fr[u] ^ 1] += fl[t]; // 从 T 一路反推到 S, 并更新每条边的剩余流量
            }
        }
    }
} g;
int n, m, s, t;
int main() {
    cin >> n >> m >> s >> t;
    for(int i = 1; i <= m; i++) {
        int u, v, w;
        cin >> u >> v >> w, g.add(u, v, w);
    }
    cout << g.maxflow(s, t) << endl;
    return 0;
}

```

复杂度证明

为证明 EK 算法的时间复杂度，我们需要这样一条引理：

引理

每次增广后残量网络上 S 到每个点的最短路长度 **不减**。

证明

考虑从 S 出发的最短路 DAG 的分层图。新加入的边一定是从 dis 较大的点指向 dis 较小的点。

□

设原残量网络的最短路长度为 dis 。

如果最短路长度减小了，那么在新的最短路上，总存在一条边 $x \rightarrow y$ 使得 $dis_x + 1 < dis_y$ 。这条边不可能是原残量网络的边，否则就不满足三角形不等式了。所以 $x \rightarrow y$ 一定是新加入的边，即 $y \rightarrow x$ 被增广。一条边被增广需要它在最短路上，所以

$$dis_y + 1 = dis_x < dis_x + 1 < dis_y.$$

矛盾。

设增广路为 P 。根据能流满就流满的原则，存在 $x \rightarrow y$ 使得其剩余流量 $c_f(x, y)$ 等于本次增广流量 $c_f(P)$ 。于是， $x \rightarrow y$ 属于原来的残量网络，但不在增广后的残量网络上。称这种边为 **关键边**。

因为增广路是最短路， $dis_x + 1 = dis_y$ 。在使得 $x \rightarrow y$ 再次出现在残量网络上的增广之前， $dis'_y + 1 = dis'_x$ 。所以

$$dis_x + 1 = dis_y \leq dis'_y = dis'_x - 1.$$

因此 $x \rightarrow y$ 每次在残量网络上消失又出现的过程必然使得 dis_x 至少增加 2。

综上，每条边作为关键边的次数不超过 $\mathcal{O}(n)$ 。因为一次增广必然存在关键边，所以增广次数不超过 $\mathcal{O}(nm)$ ，时间复杂度 $\mathcal{O}(nm^2)$ 。

注

从复杂度分析的过程来看，这个上界是很宽松的。虽然总能构造特殊图卡满上界（同阶），但常数非常小。

2.4 Dinic 算法

算法介绍

Dinic 算法的核心思想是 **分层图** 和 **相邻层之间增广**。首先 BFS 得到分层图，然后从 S 出发进行 DFS **多路增广**。维护当前点的剩余流量，向下一层的点继续流。

给图分层，将本次增广限制在 DAG 上的目的是规范增广路的形态，防止增广路成环。

当前弧优化 保证了 Dinic 算法的时间复杂度。增广时，已经流满的边没有用，可以直接跳过，不需要每次都从邻接表头开始遍历。为此，记录从每个点出发的第一条没有流满的边，称为 **当前弧**。搜索到一个点时，从其当前弧开始增广。

注意

每次多路增广前每个点的当前弧应初始化为邻接表头。一条边并非一旦流满就无用。反向边流量的增加会让它重新出现在残量网络中。

当前弧优化后的 Dinic 算法的时间复杂度 $\mathcal{O}(n^2m)$ 。若不加当前弧优化，时间复杂度会退化成和 EK 一样的 $\mathcal{O}(nm^2)$ 。此时由于 DFS 常数过大，实际表现并没有 EK 优秀。

注

Dinic 实际上蕴含了 EK，因为 Dinic 本质也是找最短路增广。相较于 EK，Dinic 加入了多路增广和当前弧优化。

当前弧优化的注意点

```
for(int i = cur[u]; res && i; i = nxt[i]) {
    cur[u] = i;
    // do something
}
```

上述代码不可以写成

```
for(int &i = cur[u]; res && i; i = nxt[i]) {
    // do something
}
```

若 $u \rightarrow v$ 这条边让剩余流量 `res` 变成 0，第二种写法会使 u 的当前弧直接跳过 $u \rightarrow v$ 。但 $u \rightarrow v$ 不一定流满，所以不应跳过。这会导致当前弧跳过很多未流满的边，使增广效率降低，从而大幅降低程

序运行效率。实际表现比 E-K 还要差。

另一种解决方法是在循环末尾判断 `if(!res) return flow;`。总之，在写当前弧优化时注意不能跳过没有流满的边。

模板题 代码如下。


```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const int N = 200 + 5, M = 5e3 + 5;
struct flow {
    int cnt = 1, hd[N], nxt[M << 1], to[M << 1], limit[M << 1];
    void add(int u, int v, int w) {
        nxt[++cnt] = hd[u], hd[u] = cnt, to[cnt] = v, limit[cnt] = w;
        nxt[++cnt] = hd[v], hd[v] = cnt, to[cnt] = u, limit[cnt] = 0;
    }
    int T, dis[N], cur[N];
    ll dfs(int id, ll res) {
        if(id == T) return res;
        ll flow = 0;
        for(int i = cur[id]; i && res; i = nxt[i]) {
            cur[id] = i;
            int c = min(res, ll(limit[i])), it = to[i];
            if(dis[id] + 1 == dis[it] && c) {
                int k = dfs(it, c);
                flow += k, res -= k;
                limit[i] -= k, limit[i ^ 1] += k;
            }
        }
        if(!flow) dis[id] = -1;
        return flow;
    }
    ll maxflow(int s, int t) {
        T = t;
        ll flow = 0;
        while(1) {
            queue<int> q;
            memcpy(cur, hd, sizeof(hd));
            memset(dis, -1, sizeof(dis));
            q.push(s), dis[s] = 0;
            while(!q.empty()) {
                int t = q.front();
                q.pop();
                for(int i = hd[t]; i; i = nxt[i]) {
                    if(dis[to[i]] == -1 && limit[i]) {
                        dis[to[i]] = dis[t] + 1, q.push(to[i]);
                    }
                }
            }
            if(dis[t] == -1) return flow;
            flow += dfs(s, 1e18);
        }
    }
} g;
int n, m, s, t;
int main() {
    cin >> n >> m >> s >> t;

```

```

for(int i = 1; i <= m; i++) {
    int u, v, w;
    cin >> u >> v >> w, g.add(u, v, w);
}
cout << g.maxflow(s, t) << endl;
return 0;
}

```

复杂度证明

在证明 EK 的时间复杂度时，我们用到了结论： S 到每个点的最短路单调不减。因为 Dinic 蕴含 EK，所以该引理仍然成立。

考虑增广轮数和一轮增广的复杂度。

结论

Dinic 的每次增广都会使得 S 到 T 的最短路长度增加。

证明

反证法。假设存在一次增广使得 S 到 T 的最短路长度没有增加。由引理， S 到 T 的最短路不变。

由一轮增广的过程， S 到 T 的最短路 P 不可能是原残量网络上的最短路（否则会被增广掉）。于是存在 $(x, y) \in P$ 使得 $dis_x + 1 < dis_y$ 。类似引理的证明可以导出矛盾。□

于是增广轮数为 $\mathcal{O}(n)$ 级别。

DFS 时，每次到达 T 都代表找到一条增广路。我们将寻找这条增广路的代价放大至增广路的长度，即 $\mathcal{O}(n)$ 。实际远达不到这一上界（但仍然是 $\mathcal{O}(n)$ ，达不到上界指常数非常小）。

找到增广路后，我们将不断回溯，直到当前点还有多余的流量。那么增广路上从这个点出发的那条边（也就是关键边）将满流。因此增广路的数量不超过 $\mathcal{O}(m)$ 。

综上，一次增广的复杂度为 $\mathcal{O}(nm)$ 。可以看出这个上界非常松，这也是为什么 Dinic 算法在求网络最大流时表现良好。

3. 无负环的费用流

费用流一般指 **最小费用最大流** *minimum cost maximum flow*, MCMF。

在原网络 G 的基础上，每条边有 **权值** $w(x, y)$ 。最小费用最大流要求保证最大流时 $\sum_{(x,y) \in E} f(x, y) \times w(x, y)$ 的最小值。 w 即每条边流单位流量的费用，我们需要在最大流的前提下最小化费用，因此该问题称为费用流。

本章讨论 **初始残量网络无负环** 的费用流。关于有负环的费用流，见 6.2 小节。

3.1 SSP 算法

算法介绍

连续最短路 *successive shortest path*，简称 SSP。该算法的核心思想是每次找到 **长度最短的增广路** 进行增广。仅在网络 **初始无负环** 时能得到正确答案。

SSP 算法有两种实现方法，一种基于 EK，另一种基于 Dinic。这两种方法均需要将 BFS 换成 SPFA（每条边的长度为 w ）求最短路，且 Dinic 的 DFS 多路增广仅在满足 $dis_x + w(x, y) = dis_y$ 的边上进行。

注意

在使用 SPFA 求 S 到 T 的最短路时，不能在队头为 T 时就退出算法，因为此时 $dis[T]$ 不一定取到最短路。

退流 $x \rightarrow y$ 时，其费用也要退掉。因此，对原图的每条边 $x \rightarrow y$ ，其在残量网络上的反边的权值 $w(y, x)$ 为 $-w(x, y)$ 。

注意

原图上的一条边和它的反边是独立的，它们在残量网络上有各自独立的反边（见基本定义最后一段），所以权值不冲突。

SSP 算法的时间复杂度为 $\mathcal{O}(nmf)$ ，其中 f 是最大流流量，这说明 SSP 算法是伪多项式时间的。在实际应用中，该上界非常宽松。不仅增广次数远远达不到 f ，同时 SPFA 的时间也远远达不到 $\mathcal{O}(nm)$ ，可以放心使用。

注

一般以 Dinic 作为网络最大流的标准算法，以基于 EK 的 SSP 作为费用流的标准算法。

「最大流不卡 Dinic，费用流不卡 EK」是业界公约。

[模板题](#) 代码。

```

#include <bits/stdc++.h>
using namespace std;
const int N = 5e3 + 5, M = 5e4 + 5;
struct flow {
    int cnt = 1, hd[N], nxt[M << 1], to[M << 1], limit[M << 1], cst[M << 1];
    void add(int u, int v, int w, int c) {
        nxt[++cnt] = hd[u], hd[u] = cnt, to[cnt] = v, limit[cnt] = w, cst[cnt] = c;
        nxt[++cnt] = hd[v], hd[v] = cnt, to[cnt] = u, limit[cnt] = 0, cst[cnt] = -c;
    }
    int fr[N], fl[N], in[N], dis[N];
    pair<int, int> mincost(int s, int t) {
        int flow = 0, cost = 0;
        while(1) { // SPFA
            queue<int> q;
            memset(dis, 0x3f, sizeof(dis));
            memset(in, 0, sizeof(in));
            fl[s] = 1e9, dis[s] = 0, q.push(s);
            while(!q.empty()) {
                int t = q.front();
                q.pop(), in[t] = 0;
                for(int i = hd[t]; i; i = nxt[i]) {
                    int it = to[i], d = dis[t] + cst[i];
                    if(limit[i] && d < dis[it]) {
                        fl[it] = min(limit[i], fl[t]), fr[it] = i, dis[it] = d;
                        if(!in[it]) in[it] = 1, q.push(it);
                    }
                }
            }
            if(dis[t] > 1e9) return make_pair(flow, cost);
            flow += fl[t], cost += dis[t] * fl[t];
            for(int u = t; u != s; u = to[fr[u] ^ 1]) {
                limit[fr[u]] -= fl[t], limit[fr[u] ^ 1] += fl[t];
            }
        }
    }
} g;
int n, m, s, t;
int main() {
    cin >> n >> m >> s >> t;
    for(int i = 1; i <= m; i++) {
        int u, v, w, c;
        cin >> u >> v >> w >> c, g.add(u, v, w, c);
    }
    pair<int, int> ans = g.mincost(s, t);
    cout << ans.first << " " << ans.second << endl;
    return 0;
}

```

SSP 算法的时间复杂度是显然的：一次 SPFA 是 $\mathcal{O}(nm)$ ，最多增广 f 次。

正确性证明

我们证明每次增广最短路一定能求出最小费用最大流。根据最大流最小割定理可证流的最大性，因此只需证明费用最小性。

考虑两个流量相等的可行流 f_1, f_2 ，令 $\Delta f = f_2 - f_1$ ，则 Δf 流量为零，所以 Δf 由若干个正流量的环组成。若 f_2 的费用小于 f_1 ，则 Δf 包含至少一个正流量的负环。

消圈定理

f 是所有流量与之相等的流当中费用最小的，当且仅当 f 的残量网络上不存在负环。

证明

若 f 的残量网络含负环，在负环上推流可以得到流量相等但费用更小的流。

若 f 不是费用最小的流，则存在流 f' 使得 $\Delta f = f' - f$ 包含正流负环，于是 f 的残量网络有负环。□

归纳假设增广前 f 的残量网络无负环。设增广后的流为 f' 。不妨设流量的增量为 1，则费用的增量为最短路长度。

假设存在流 f'' 的流量与 f' 相等但费用更小。因为流量增量为 1，所以 $f'' - f$ 由 S 到 T 的流量为 1 的路径 P' 和一些正流量的环 $C_1, \dots, C_k$ 组成。因为 $f' - f$ 是最短路，所以 P' 的费用不小于 $f \rightarrow f'$ 的费用。但 $f \rightarrow f''$ 的费用小于 $f \rightarrow f'$ 的费用，所以存在负环 C_i ，矛盾。

因此，不存在流量与 f' 相等但费用更小的流。由消圈定理， f' 的残量网络上无负环。因为初始网络无负环，所以归纳法成立。□

3.2 Primal-Dual 算法

建议先学习 Johnson 全源最短路算法，详见 [图论 I](#)。

注

Primal-Dual 原始对偶实际上是一个重要的组合优化技术。但是笔者并没有看出以下算法和它的关联。

和 SSP 一样，Primal-Dual 仅适用于无负环的网络。

算法尝试为每个点赋势能 h_i ，保持原图最短路不变的同时让边权非负。这个目标和 Johnson 全源最短路的第一步如出一辙。

首先使用 SPFA 求出源点到每个点的最短路 h_i ，则 $i \rightarrow j$ 的新边权为 $w'_{i,j} = w_{i,j} + h_i - h_j$ 。根据三角形不等式， $w'_{i,j} \geq 0$ 。因此，经过转化，我们可以使用更稳定的 Dijkstra 求增广路。

每次增广都会改变残量网络的形态。为此，用每次增广时跑出来的最短路加在 h 上，即 $h'_i \leftarrow h_i + dis_i$ 。

正确性证明

如果 $i \rightarrow j$ 在增广路上，有 $dis_i + w_{i,j} + (h_i - h_j) = dis_j$ 。因为 $w_{i,j} = -w_{j,i}$ ，所以 $w_{j,i} + (dis_j + h_j) - (dis_i + h_i) = 0$ ，即新增的反边边权为 0。

对于原有的边， $dis_i + w_{i,j} + (h_i - h_j) \geq dis_j$ ，即 $w_{i,j} + (dis_i + h_i) - (dis_j + h_j) \geq 0$ ，原边权仍然非负。

□

实际表现方面，Primal-Dual 相较于 SSP 并没有很大的优势，可能是因为 SPFA 本身已经够快了。而且 Primal-Dual 写起来没有 SSP 方便。

代码。

4. 上下界网络流

在原网络 G 的基础上，每条边有 **流量下界** $b(u, v)$ 。这使得可行的流函数应满足的流量限制更加严格： $b(u, v) \leq f(u, v) \leq c(u, v)$ 。

4.1 无源汇可行流

无源汇上下界可行流是上下界网络流的基础。我们需要为一张无源汇的网络寻找一个流函数 f ，使其满足流量限制，斜对称以及流量守恒。解决这个问题的核心思想，是 **先满足流量下界的限制，再尝试调整**。

具体地，我们首先让每条边 (u, v) 都 **流满下界** $b(u, v)$ ，并算出每个点的 **净流量** w_i ，即流入 i 的总流量减去从 i 流出的总流量。

当 $w_i > 0$ 时，说明当前流到 i 的流量太多了，还要再流出去 w_i 才能守恒。相反，若 $w_i < 0$ ，说明 i 还要流进 $-w_i$ 单位流量。根据斜对称性，有 $\sum w_i = 0$ 。由此可设 $\Delta = \sum_{w_i > 0} w_i = \sum_{w_i < 0} -w_i$ 。

这启发我们新建无容量下界的网络 $G' \approx G$ ，每条边的流量限制 $c' = c - b$ ，并认为最终的可行流 f 为 b 加上 G' 上的可行流 f' 。

新建独立于原有点集的超级源点 SS 和 超级汇点 TT (虽然 G 无源无汇, 但这是为了区分有源汇时不同最大流过程的源点和汇点)。若 $w_i > 0$, 则连边 $SS \rightarrow i$ 容量为 w_i , 否则连边 $i \rightarrow TT$ 容量为 $-w_i$ 。不难发现, 从 SS 连出了总容量为 Δ 的边, 且总容量为 Δ 的边连向了 TT 。

注意

初学者容易搞反连边的方式。我们可能会认为, 如果 $w_i > 0$, 即流到 i 的流量太多了, 那应该连边 $i \rightarrow TT$ 容量为 w_i , 把这些流量流走才对。原因是最终可行流的定义 $f = b + f'$ 。在 b 中 i 的净流量为 w_i , 所以 f' 在原图上的那部分 (即去掉 i 和超源或超汇的连边) 的净流量应当为 $-w_i$ 。为了让 f' 本身是流量守恒的, 需连边 $SS \rightarrow i$ 补充 w_i 流量。

换言之, SS 和 TT 的连边起到了 **模拟** 每个点的净流量的作用, 而非补充或消耗。

为了让每个点的净流量变为 0, 所有超源和超汇相关的连边必须流满。于是, 若 SS 到 TT 的最大流 f' 的流量不等于 Δ , 说明找不到一种合法方案使得同时满足流量限制和流量守恒, 问题无解。相反, 若最大流等于 Δ , 则将 f' 的流量加在 b 上, 得到可行流 $f = b + f'$ 。

- 因为 SS 到 TT 满流, 所以 f 的每个点的净流量均为 0。
- 因为 G' 上的流量限制是在每条边在 b 的基础上还能流多少, 所以 f 满足流量限制, 即 $f = f' + b \leq c' + b = c$ 。

代码。

4.2 有源汇可行流

连边 $T \rightarrow S$ 容量为 $+\infty$, 转化为无源汇上下界可行流。

4.3 有源汇最大流

有源汇上下界最大流基于一个非常关键的结论: 给定任意一组 **可行流**, 对其运行 EK 或 Dinic 等基于增广操作的最大流算法, 我们总能得到正确的最大流。因为增广本身支持退流, 所以无论初始的流函数 f 如何, 只要 f 合法, 就一定能求出最大流。

因此, 我们考虑先求出任意一组可行流, 再进行调整: 首先对 G 求 5.2 小节的有源汇 **可行流**, 过程中我们会新建网络 G' 。撤去 SS, TT 以及 $T \rightarrow S$ 的容量为 $+\infty$ 的边。 SS, TT 的意义是求无源汇可行流, $T \rightarrow S$ 的意义是将有源汇可行流转化为无源汇可行流, 而我们已经得到了一组有源汇可行流, 除非转化的无源汇可行流问题无解。当前可行流流量即 $T \rightarrow S$ 的流量, 即残量网络上的反边的容量。

注意

可行流流量是 $T \rightarrow S$ 的反边流量, 而不是 SS 到 TT 的流量。

根据开头给出的结论，我们只需要以 S 为源， T 为汇在 G' 上跑最大流，并将可行流流量与最大流流量（新增流量）相加即为有源汇最大流。注意与 SS, TT 作区分。

注意

二次调整在 G' 上进行，不能在 G 上面跑最大流，因为 G 的退流操作会使得 f 不符合容量限制，但 G' 的退流操作不会。

因为 G 的实际流量 f 等于 $b + f'$ ，所以只要 f' 符合容量限制，那么 f 也一定符合。

代码。

4.4 有源汇最小流

根据 S 到 T 的最小流等于 $T \rightarrow S$ 的最大流的相反数这一结论，用可行流减掉 G' 上 $T \rightarrow S$ 的最大流。

代码。

4.5 无源汇费用流

将最大流算法换成费用流，所有和 SS, TT 相关的连边代价均为 0。因为 G' 相较 G 只增加了 SS 和 TT 的边，所以 G' 无负环当且仅当 G 无负环。

初始费用为 $\sum b(x, y)w(x, y)$ 。需加上调整净流量所产生的费用，即 SS 到 TT 的最小费用最大流的费用。

4.6 有源汇费用流

注意连边 $T \rightarrow S$ 可能导致网络出现负环，所以通常使用下一小节的技巧消去图上的负权边。

对于有源汇最小费用最大流，在有源汇最小费用可行流的基础上，进行二次调整时也要加上产生的费用。特别地，二次调整前删去 SS, TT 和 $T \rightarrow S$ 不会让残量网络出现负环。

4.7 有负环的费用流

有负环不影响最小费用存在，因为有流量限制。

注

对于有源汇的情况，允许成环的流量不经过 S 和 T ，否则可以“挑战哈密顿”：所有边的费用为 -1 。建超源 SS 连边 $SS \rightarrow S$ 容量为 1 且费用为 0 以限制流量为 1，则 S 到 T 存在哈密顿路

径当且仅当最小费用最大流的费用为 $1 - n$ 。

考虑上下界网络流，强制令负权边满流，加入反边 $b(y, x) = 0$, $c(y, x) = c(u, v)$ 表示退流。此时，负权边的流量等于容量，所以不会出现在 G' 中，初始残量网络无负环。使用上一小节的上下界费用流算法即可。

称为 **消圈算法**。

最大费用最大流：将边权取反，求可能有负环的费用流。

模板题 代码如下。

```

#include <bits/stdc++.h>
using namespace std;
const int N = 200 + 5, M = 2e4 + N;
struct flow {
    int cnt = 1, hd[N], nxt[M << 1], to[M << 1], limit[M << 1], cst[M << 1];
    void add(int u, int v, int w, int c) {
        nxt[++cnt] = hd[u], hd[u] = cnt, to[cnt] = v, limit[cnt] = w, cst[cnt] = c;
        nxt[++cnt] = hd[v], hd[v] = cnt, to[cnt] = u, limit[cnt] = 0, cst[cnt] = -c;
    }
    int fl[N], fr[N], dis[N], in[N];
    pair<int, int> mincost(int s, int t) {
        int flow = 0, cost = 0;
        while(1) {
            queue<int> q;
            memset(dis, 0x3f, sizeof(dis));
            q.push(s), fl[s] = 1e9, dis[s] = 0;
            while(!q.empty()) {
                int t = q.front();
                q.pop(), in[t] = 0;
                for(int i = hd[t]; i; i = nxt[i]) {
                    int it = to[i], d = dis[t] + cst[i];
                    if(limit[i] && d < dis[it]) {
                        dis[it] = d, fl[it] = min(fl[t], limit[i]), fr[it] = i;
                        if(!in[it]) in[it] = 1, q.push(it);
                    }
                }
            }
            if(dis[t] > 1e9) return make_pair(flow, cost);
            flow += fl[t], cost += dis[t] * fl[t];
            for(int u = t; u != s; u = to[fr[u] ^ 1]) {
                limit[fr[u]] -= fl[t], limit[fr[u] ^ 1] += fl[t];
            }
        }
    }
};

struct bounded_flow {
    int e, u[M], v[M], lo[M], hi[M], cst[M];
    void add(int _u, int _v, int w, int c) {
        if(c < 0) {
            u[++e] = _u, v[e] = _v, lo[e] = w, hi[e] = w, cst[e] = c;
            u[++e] = _v, v[e] = _u, lo[e] = 0, hi[e] = w, cst[e] = -c;
        }
        else u[++e] = _u, v[e] = _v, lo[e] = 0, hi[e] = w, cst[e] = c;
    }
    flow g;
    pair<int, int> mincost(int n, int s, int t, int ss, int tt) {
        static int w[N];
        memset(w, 0, sizeof(w));
        int flow = 0, cost = 0, tot = 0;
        for(int i = 1; i <= e; i++) {
            w[u[i]] -= lo[i], w[v[i]] += lo[i];

```

```

    cost += lo[i] * cst[i];
    g.add(u[i], v[i], hi[i] - lo[i], cst[i]);
}
for(int i = 1; i <= n; i++) {
    if(w[i] > 0) g.add(ss, i, w[i], 0), tot += w[i];
    else if(w[i] < 0) g.add(i, tt, -w[i], 0);
}
g.add(t, s, 1e9, 0);
pair<int, int> res = g.mincost(ss, tt);
cost += res.second;
flow += g.limit[g.hd[s]];
g.hd[s] = g.nxt[g.hd[s]], g.hd[t] = g.nxt[g.hd[t]];
res = g.mincost(s, t);
return make_pair(flow + res.first, cost + res.second);
}
} f;
int n, m, s, t;
int main() {
    cin >> n >> m >> s >> t;
    for(int i = 1; i <= m; i++) {
        int u, v, w, c;
        cin >> u >> v >> w >> c, f.add(u, v, w, c);
    }
    pair<int, int> res = f.mincost(n, s, t, 0, n + 1);
    cout << res.first << " " << res.second << endl;
    return 0;
}

```

4.8 总结

先满足流量下界的限制，转化成无下界限制的情形，再调整。

无源汇可行流：最基本的情况，算出每个点的净流量后最大流调整。

有源汇可行流：加入 $T \rightarrow S$ 转化为无源汇可行流。

有源汇最大流：先求可行流，在删去 SS, TT 和 $T \rightarrow S$ 的 G' 上跑 S 到 T 的最大流。

有源汇最小流：先求可行流，在删去 SS, TT 和 $T \rightarrow S$ 的 G' 上跑 T 到 S 的最大流。

若带有费用，则将最大流换成最小费用最大流。通过取反将最大费用转化为最小费用。需要保证运行费用流算法的网络 (G') 无负环，可以用消圈算法除去所有负权边。

5. 对网络流的理解

网络流的本质是反悔贪心。解决网络流问题分成两步：相信问题可以用网络流算法解决，建图。

5.1 网络流与贪心

在网络流算法中，我们的策略是贪心地找到最短路进行增广。但这样的决策不一定最优，需要为反边添加容量。流过反边即退掉流量，对应原问题的 **反悔** 操作。因此，网络流本质上是 **可反悔贪心**。运用上下界网络流等技巧可以方便地处理问题的限制。

更一般地，网络流也是一种特殊的贪心，它们之间可以相互转化。对于具有特定增广模式（网络具有某种性质）的网络流，可以从贪心的角度考虑，从而使用数据结构维护，称为 **模拟费用流**。很多贪心问题也可以通过网络流解释。

换言之，网络流 **将贪心以图的形式刻画**。解决网络流问题的算法与某个支持反悔的贪心策略相对应，使得我们不需要为每个贪心问题都寻找反悔策略。相反，建出图后就是一遍最大流或费用流的事了。

注意

因为网络流算法本身自带反悔操作，所以在解决动态加边的 **最大流** 问题时，我们不需要担心原来的流方案会影响到算法求解新图最大流时的正确性。

但对于费用流，因为其正确性依赖于每一步增广路均为最短路，所以一旦给网络加入新边，就必须重新跑费用流才能得到正确费用。

5.2 网络流的技巧

解决可以转化为网络流的最优化问题，关键在于如何建图，将问题的限制转化为图上的边，使得 **问题的每种方案都与一种流或割对应**。最大化问题考虑最大流或转化为最小割，最小化问题考虑最小割。

例如，在 **P2057** 一题中，直接对每个小朋友拆点并不可行，因为无法考虑到他与他的朋友意见不一致时的代价。题目要求最小化代价，所以考虑用一组割表示一种意见方案。一个点在割中要么属于 A ，要么属于 B ，正好刻画了一个小朋友要么支持，要么反对。不妨认为属于 A 表示支持，属于 B 表示反对。

- 如果 i 原本支持但 $i \in B$ ，则有 1 的代价。如何让 $i \in B$ 有 1 的代价？连边 $S \rightarrow i$ 容量为 1。 $i \in B$ 说明 $S \rightarrow i$ 被割掉，产生 1 的代价。
- 类似地，如果 i 原本反对，则连边 $i \rightarrow T$ 容量为 1， $i \in A$ 说明 $i \rightarrow T$ 被割掉，产生 1 的代价。对应 i 原本反对但 $i \in A$ ，意愿不同，有 1 的代价。
- 如何刻画投票不同的好朋友的代价？若 $i \in A$ ， $j \in B$ 且 i, j 互为好朋友，则有 1 的代价。因为 $i \in A$ ， $j \in B$ ，所以 $i \rightarrow j$ 若存在则一定是割边。连边 $i \rightarrow j$ 容量为 1 即可。

对于一组割，其唯一对应了一个投票方案。残量网络上属于 A 的人支持，属于 B 的人反对。这是经典的 **集合划分模型**。

5.3 求方案时的注意点

在尝试根据最大流求最小割时，初学者可能会陷入一个误区，认为满流的边即割边。这显然是错误的，反例如 $c(S, 1) = c(1, T) = 1$ 。

回忆割的定义：将 V 分成两个互不相交的点集 A, B 满足 $S \in A$ 且 $T \in B$ ，则所有 $A \rightarrow B$ 的边才是割边。最大流最小割的证明自然地给出了一组割：若 S 可达 u ，则 $u \in A$ ，否则 $u \in B$ 。

因此，若一组割对应了题目的一种方案，在求出最大流之后，不能将流满的边视作割边，而是将两端所在集合不同的边视作割边。在使用集合划分模型求方案时需格外注意。

6. 常见模型和问题

其实是常见建图套路。

6.1 最小割点

题目描述

给定删去每个点的代价 w_i ，求使得 S, T 不连通的最小代价。

之前的最小割都是割边。

点边转化 是网络流的常用技巧。将每个点拆成入点 i_{in} 和出点 i_{out} ，连边 $i_{in} \rightarrow i_{out}$ 容量为 w_i ，表示删去这个点使得 i_{in} 与 i_{out} 不连通需要 w_i 的代价。对原图的每一条边 (u, v) ，连边 $u_{out} \rightarrow v_{in}$ 容量为 $+\infty$ ，表示只能删点，不能删边。

不难发现 S_{out} 到 T_{in} 的最小割即为所求。

例题：P1345, P3356, P2045, P2153。

*6.2 集合划分模型

集合划分模型是网络流常见模型。这个模型应用广泛，可以加深对最小割问题的理解，读者应充分掌握。

问题描述

有 n 个物品， A, B 两个集合。物品 i 分到集合 A 有代价 a_i ，分到集合 B 有代价 b_i 。给定 m 条限制 (u_j, v_j, c_j) ，表示若 $u_j \in A$ 且 $v_j \in B$ ，则有代价 c_j 。求最小代价。

复杂度只能和 n, m 有关。

注

一种等价形式为

$$\min_{x_1 \sim n} \left(\left(\sum_{u=1}^n a_u x_u \right) + \left(\sum_{u=1}^n b_u \overline{x_u} \right) + \left(\sum_{(u,v) \in E} c_{u,v} x_u \overline{x_v} \right) \right),$$

其中 x_i 等于 0 或 1, $\overline{x_i}$ 表示 x_i 取反。 $x_i = 1$ 表示分到 A , $x_i = 0$ 表示分到集合 B 。

给定 a, b, c, E , 为 x_i 选择合适的值最小化整个和式。

建出网络:

- 连边 $S \rightarrow i$ 容量为 b_i , 表示若 $i \in B$ 则 $S \rightarrow i$ 被割掉, 代价为 b_i 。
- 连边 $i \rightarrow T$ 容量为 a_i , 表示若 $i \in A$ 则 $i \rightarrow T$ 被割掉, 代价为 a_i 。
- 连边 $u_j \rightarrow v_j$ 容量为 c_j , 表示若 $u_j \in A$ 且 $v_j \in B$ 则 $u_j \rightarrow v_j$ 被割掉, 代价为 c_j 。

$S \rightarrow i$ 和 $i \rightarrow T$ 只会根据 $i \in A$ 或 $i \in B$ 被割掉恰好一个。对上述网络求最小割即可。

讨论一些扩展问题:

- 当 a_i, b_i 出现负值时, 直接最小割转最大流不能得到正确结果, 因为难以解决容量为负的最大流问题。考虑将 a_i, b_i 同时加上 δ_i , 最后在求出的最小割中减掉 $\sum \delta_i$ 。因为 i 要么属于 A , 要么属于 B , 所以同时为 a_i, b_i 加上 δ_i 对最小割的影响为 δ_i 。体现在图上即 $S \rightarrow i \rightarrow T$, 为了使 S, T 不连通, 必须 **至少割掉一条边**。同时, 我们也只会 **恰好割掉一条边**, 否则和割的定义矛盾。
- 当 c_j 出现负值时, 除非所有 c 均为负且要求最大化“贡献”(此时所有边权取反并最小化代价, 包括 a 和 b), 否则问题难以解决。取反可以从 **代价和贡献的转化** 的角度来理解, 即若代价为 1, 则贡献为 -1 。一般我们希望最大化贡献, 最小化代价。所以在建图时, 我们尽量从“**若 则有 的代价**”的角度思考问题, 会更方便(例如“若 则不满足题目限制, 有 $+\infty$ 的代价”)。
- 若题目要求输出方案, 见 5.3 小节。

集合划分模型的单边联合代价 (文理分科模型)

在集合划分模型的基础上, 我们再增加一些条件: 给出若干集合 S_k 和权值 $e_k \geq 0$, 若并非所有 $i \in S_k$ 均属于 A , 则有代价 e_k 。

条件可等价表述为存在 $j \in S_k$ 属于 B , 因此新建表示限制的点 v_k , $S \rightarrow v_k$ 容量 e_k , $v_k \rightarrow i$ 容量 $+\infty$ 。若存在 $i \in S_k$ 被划分至 B , 则 $S \rightarrow v_k$ 是割边, 代价为 e_k 。

对于同属于右部限制，同理。这种建图方式俗称“文理分科模型”，由经典题“文理分科”得名。

6.3 最大权闭合子图

闭合子图：有向图 $G = (V, E)$ 的闭合子图 G' 定义在点集 $V' \subseteq V$ 上。 V' 合法当且仅当 V' 内部每个点的所有出边仍指向 V' ，即点集内部每个点在有向图上能够到达的点仍属于该点集。 V' 的点导出子图（即 V' 和所有 V' 内部的边）即 G' 。

问题描述

给定有向图 G ，每个点有点权 w_i 。子图的权值为点集内每个点的权值之和，求闭合子图的最大权值。

复杂度只能和 n, m 有关。

首先将最大化转化为最小化（最大割是 NP-Hard 问题）。默认选择所有点，不选点 i 有 w_i 的代价，则最大权值为所有点的权值和减去最小代价。

考虑集合划分模型。对每个点，将其划分到 **选** 或 **不选** 的集合中。 $i \in B$ 表示不选，割掉 $S \rightarrow i$ ，代价为 w_i 。因此 $S \rightarrow i$ 有容量 w_i 。同理， $i \rightarrow T$ 有容量 0。如果 $(u, v) \in E$ ，说明若 u 分到选的集合中，则 v 也必须分到选的集合当中，即 $u \in A, v \in B$ 有代价 $+\infty$ 。

w_i 有负值（否则选整张图即可），使用上一小节讨论的解决方法，若 $w_i < 0$ ，则给 $S \rightarrow i$ 和 $i \rightarrow T$ 同时加上 $-w_i$ ，即默认选择 i ，如果不选有 $-w_i$ 的代价。

将两步转化结合在一起，得到解法：先将所有正权点选入闭合子图，再去掉不选的正权点的贡献。最小化去掉的正权点的代价 w_i 以及选择负权点的代价 $-w_i$ 之和。

综上，我们得到了最大权闭合子图的一般解法：

- 对于 $w_i > 0$ ，连边 $S \rightarrow i$ 容量为 w_i ，表示若 $i \in B$ 即不选 i ，则 $S \rightarrow i$ 被割掉，代价为 w_i 。
- 对于 $w_i < 0$ ，连边 $i \rightarrow T$ 容量为 $-w_i$ ，表示若 $i \in A$ 即选 i ，则 $i \rightarrow T$ 被割掉，代价为 $-w_i$ 。
- 对于 $(u, v) \in E$ ，连边 $u \rightarrow v$ 容量为 $+\infty$ ，表示不可以 $u \in A$ 且 $v \in B$ ，即不可以选 u 但不选 v 。
- 设得到网络 G' 。最终答案 $(\sum_{w_i > 0} w_i) - \text{MinCut}(G')$ 。

例题：P2057，P2762，P2805。

6.4 切糕模型

问题描述

n 个整数变量 $x_i \in [1, V]$, $x_i = c$ 的代价为 $f_{i,c}$ 。给定 m 条限制 (u_j, v_j, d_j) , 表示 $x_{u_j} \leq x_{v_j} + d_j$ 。求满足所有限制的最小代价。

复杂度可以和 n, m, V 有关, 但必须是多项式级别。

用流量表示变量的取值显然不太可行, 因为连代价都无法刻画, 更不用提限制了。所以考虑最小割。

考虑将每个点拆成长为 V 的链 $x_{i,0} \rightarrow \cdots \rightarrow x_{i,V}$, 用割掉 $x_{i,c-1} \rightarrow x_{i,c}$ 表示 $x_i = c$, 所以 $x_{i,c-1} \rightarrow x_{i,c}$ 的容量为 $f_{i,c}$ 。

如何限制一条链上只能割一条边呢? 一条链上所有属于 A 的点是一段前缀: 若 $x_{i,c} \in A$, 则所有 $x_{i,c' < c}$ 都应当属于 A 。这启发我们连边 $x_{i,c} \rightarrow x_{i,c' < c}$ 容量为 $+\infty$ 。边数有点多, 但这显然等价于连边 $x_{i,c} \rightarrow x_{i,c-1}$ 容量为 $+\infty$ (前缀优化建图)。

这样, 一条链只能割一条边, 且 $x_i \geq c$ 当且仅当 $x_{i,c-1} \in A$ 。

接下来刻画限制 $x_u \leq x_v + d$ 。 $x_v \geq x_u - d$ 等价于对任意 c , 若 $x_u \geq c$, 则 $x_v \geq c - d$, 即若 $x_u \geq c$, 则 $x_v < c - d$ 将产生 $+\infty$ 的代价。这启发我们对每个 $1 \leq c \leq V$ 连边 $x_{u,c-1} \rightarrow x_{v,c-d-1}$ 容量为 $+\infty$ 。

网络的点数为 $\mathcal{O}(nV)$, 边数为 $\mathcal{O}((n+m)V)$ 。求最小割即可。

这种建图方式称为“切糕模型”, 由经典题“切糕”得名。

注

这个问题也可以从集合划分模型的角度理解:

- $x_{i,c} \in A$ 表示 $x_i \geq c$ 。 $x_{i,c} \in B$ 表示 $x_i < c$ 。
- 对 $c' < c$, 若 $x_i \geq c$, 则不能有 $x_i < c'$ 。于是连边 $x_{i,c} \rightarrow x_{i,c'}$ 容量为 $+\infty$, 用前缀优化建图。
- 对 $x_u \leq x_v + d$, 若 $x_u \geq c$, 则不能有 $x_v < c - d$ 。于是连边 $x_{u,c-1} \rightarrow x_{v,c-d-1}$ 容量为 $+\infty$ 。
- 若 $x_i = c$, 则恰有所有 $x_{i,c' \leq c} \in A$ 。将代价摊到前缀的每个点上, $x_{i,c} \in A$ 的代价为 $f_{i,c} - f_{i,c-1}$ 。

注

当 $d \geq 0$ 时, 流量为 $+\infty$ 的反向链是不必要的。

假设存在一条链割掉了两条边, 考虑“中间链”, 左端属于 B , 右端属于 A 。

右端属于 A 说明 S 在残量网络上可达右端。考虑从右端在残量网络上走反边回到 S ，那么经过的所有点都属于 A 。因为左端属于 B ，所以过程中一定会经过某个 (u, v, d) 限制对应的边走到另外一条链上。如果走到的是另外一条链的“中间链”，那么不断重复该过程，直到走到某个链的“左部链”，即第一条被割掉的边左边的链。

假设从中间链的 $x_{i,c}$ 走到左部链的 $x_{i',c'}$ ，那么原图有边 $x_{i',c'} \rightarrow x_{i,c}$ 。根据连边的方式，将 c 和 c' 减去相同的值，连边依然存在。因为 $d \geq 0$ ，所以 $c' \geq c$ ，于是左部链有一个点能走到中间链的左端，和左端属于 B 矛盾。

如果 d 可能小于 0，那么为了走到中间链的左端，可能需要左部链的下标变成负数。解决方法是 S 向 i 的下标不超过 $-d$ 的点连容量 $+\infty$ 的边（将链扩展至负数下标），对应若限制 $x_{i'} \leq x_i + d$ ，则必然有 $x_i > -d$ （因为 $x_{i'} \geq 1$ ）。这样，上述证明仍然成立，使得我们不需要添加流量为 $+\infty$ 的反向链。

感谢 dengyaotriangle 的 [博客](#)。

6.5 区间选择模型

问题描述

给定 $[1, n]$ 上的 m 个区间 $[l_j, r_j]$ 。每个区间选择一次的代价为 w_j ，最多可以选 c_j 次。要求 i 被覆盖的次数在 $[a_i, b_i]$ 之间，求最小（最大）代价。

考虑用 $i \rightarrow i + 1$ 的流量描述 i 被覆盖的次数。每次选择 $[l, r]$ 都会让 l 到 $r + 1$ 上所有边被覆盖的次数增加 1。这启发我们用 $r + 1 \rightarrow l$ 表示一个区间 $[l, r]$ ，费用为 w ，容量为 c 。

这样， $r + 1 \rightarrow l$ 的每单位流量都会让 $l \rightarrow l + 1 \rightarrow \dots \rightarrow r + 1$ 的每条边的流量加 1。这是可能有负环的无源汇可行费用流。

注

用 $l \rightarrow r + 1$ 表示区间，则每单位流量都会让 $l \rightarrow l + 1 \rightarrow \dots \rightarrow r + 1$ 的每条边的流量减 1。取足够大的 X ，若 $i \rightarrow i + 1$ 的流量为 x_i ，则 i 被覆盖的次数为 $X - x_i$ 。

连边 $S \rightarrow 1$ 容量为 X 限制最大流不超过 X 。若最大流小于 X 或无解，则问题无解。

当所有点的上界相同（或无上界）时，取 X 为相同的上界，此时网络无下界限制，可以使用普通费用流（若无负环）。若最大流小于 X 则无解。

感谢 ix35 的 [博客](#)。

6.6 最大密度子图

问题描述

给定无向图 G ，求密度最大的非空点导出子图。图的密度定义为边数除以点数。

二分答案 c ，密度大于 c 等价于存在子图 G' 使得 $|E'| - c|V'| > 0$ 。考虑找到 $|E'| - c|V'|$ 最大的子图。

因为子图的构成由是否选择每条边和每个点确定，所以考虑集合划分模型。选一个点的贡献为 $-c$ ，选一条边的贡献为 1，选一条边但没选某个端点的贡献为 $-\infty$ 。若最大贡献为正，则密度大于 c ，否则密度不大于 c 。

注

要求严格大于 0 的原因是空图的权值等于 0。如果可以取等，则无法判断是否存在非空的点导出子图的权值不小于 0。

求出最大密度 d 之后还需要构造方案。为了避免求出空图，令 $d \leftarrow d - \epsilon$ 再求一遍最大贡献。若两个子图的密度不同，则至少相差 $\frac{1}{n^2}$ ，取 $\epsilon < \frac{1}{n^2}$ 即可。[模板题代码](#)。

一种更巧妙的建图方式： V' 的点导出子图的边数为每个点的度数之和减去跨过 V' 和 $V \setminus V'$ 的边数，再除以 2。所以我们只考虑每个点是否选择。选点 u 的贡献为 $d_u - 2c$ ，对 $(u, v) \in E$ 选 u 但不选 v 的贡献为 -1 。这样点数就从 $\mathcal{O}(m)$ 变成了 $\mathcal{O}(n)$ 。

6.7 二分图相关

主要有二分图最大匹配、二分图多重匹配和二分图最大权独立集。建图较简单，读者可自行尝试。

详细内容见“图的匹配”博客（以后会补充链接）。

例题：P2756，P2763，P3254，P2774，P4014，P3355。

6.8 DAG 最小路径覆盖

问题描述

给定有向无环图 G 。称路径集合 P 为 G 的路径覆盖，若 V 中的每个点 **恰好** 在 P 的一条路径上。求 G 的最小路径覆盖。

注意到表示一条边被覆盖是容易的，这启发我们使用点边转化的技巧。

将 i 拆成入点 in_i 和出点 out_i 。若 $(u, v) \in E$ ，那么连边 $out_u \rightarrow in_v$ 。

考虑建图后如何求答案。因为路径不可交，所以一个点最多有 1 入度和 1 出度，即 $S \rightarrow out$ 和 $in \rightarrow T$ 的容量为 1。

每选择一条边 (u, v) ，就相当于选中 $out_u \rightarrow in_v$ ，将 u 所在的路径和 v 所在的路径连了起来。初始路径条数为 n ，每流一条边 $out_u \rightarrow in_v$ 就相当于减少了一条路径。因此，我们希望尽可能多地匹配左右两部点。

求 S 到 T 的最大流 m ，则 $n - m$ 即最小路径覆盖的路径条数。两部点之间流满的边恰为所有被选入路径覆盖的边。设选中边集为 E' ，从 E' 上入度为 0 的点开始 DFS 即可求出每条路径。

- 若不要求路径不交（每个点至少被覆盖一次），注意到每加入一条路径所新覆盖的点形如 $u_1 \rightsquigarrow u_2 \rightsquigarrow \cdots \rightsquigarrow u_k$ ，其中 u_i 在 DAG 上可达 u_{i+1} ，于是 DAG 的最小可交路径覆盖为其传递闭包的最小不交路径覆盖。传递闭包即若 u 可达 v ，则连边 $u \rightarrow v$ 。
- 若要求每条边被覆盖至少或恰好一次，则直接在原图上求有源汇上下界最小流即可。

例题：P2764，P2765。

7. 例题

现在你已经对网络流的基本原理有了一定了解，就让我们来看一看下面这些简单的例子，把刚刚学到的知识运用到实践中吧。

7.1 网络流 24 题

LOJ6000 搭配飞行员

二分图最大匹配。

代码。

LOJ6001 太空飞行计划

最大权闭合子图。

代码。

LOJ6002 最小路径覆盖

DAG 最小路径覆盖。

代码。

LOJ6003 魔术球

一个球的上方的球的编号比它大。

若 $i < j$ 且 $i + j$ 是完全平方数，则连边 $i \rightarrow j$ 。若加入 i 时 DAG 最小路径覆盖超过 n ，则答案为 $i - 1$ 。

代码。

LOJ6004 圆桌聚餐

二分图多重匹配。

代码。

*LOJ6005 最长递增子序列

一道在网络流 24 题中不错的题目，至少笔者没有想出来（Upd on 2025.6.25：这次很快就想出来了）。

最基本的想法是在 $i < j$ 且 $a_i \leq a_j$ 的 i, j 之间连边 $i \rightarrow j$ 。但是直接这样连肯定是不行的，不能保证一定能取出最长不降子序列（LIS）。所以，我们的连边必须有一定层次。

如何给图分层呢？一个最自然且最合理的想法是按照以 a_i 结尾的 LIS 的最长长度 f_i 划分。注意到，若 $i \rightarrow j$ 有边，则一定有 $f_i < f_j$ 。又因为我们选出的链必须是 LIS，所以第 i 个点的 f 值等于 i 。

因此，为了保证选出的是 LIS，还需为连边添加 $f_i + 1 = f_j$ 的限制，并且一条链只能在 f 值等于 LIS 长度的位置停止。

通过以上分析，问题的网络流模型已经呼之欲出了。为保证一个位置只选一次，我们拆点后从入点向出点连容量为 1 的边。 S 向所有 $f_i = 1$ 的 in_i 连边，所有 $f_i = L$ 的 out_i 向 T 连边。容量均为 1。该网络的最大流流量即为第二问的答案。

对于第三问，只需将 $S \rightarrow in_1, in_1 \rightarrow out_1, in_n \rightarrow out_n$ 和 $out_n \rightarrow T$ 的容量设为 $+\infty$ 即可。这里有两个注意点，一是当 $n = 1$ 时需要特判；二是 $out_n \rightarrow T$ 在 $f_n \neq ans$ 时不存在。

代码。

LOJ6006 试题库问题

二分图多重匹配。

若最大流不等于 m 则无解，否则根据残量网络构造方案：若 $u \rightarrow v$ 的边满流，说明题目 u 属于类型 v 。

代码。

LOJ6007 方格取数

相邻格有限制的网格问题，一般通过黑白染色转化为二分图相关问题。本题就是很明显的二分图最大权独立集。

代码。

*LOJ6008 餐巾计划

拆点 是网络流的重要技巧。若仅把每天看成一个点，则干净的餐巾用完后还能作为脏餐巾继续流，导致无法区分干净的餐巾和脏餐巾。

考虑将每天拆成 i_c 和 i_d 两个点。在 i_c 层流动的是干净的餐巾，在 i_d 层流动的是脏餐巾。

- 对于购买餐巾，连边 $S \rightarrow i_c$ 容量为 $+\infty$ ，费用为 p 。
- 对于使用餐巾，连边 $i_c \rightarrow i_d$ 容量为 $+\infty$ ，流量下界为 r_i ，费用为 0。
- 对于送洗餐巾，连边 $i_d \rightarrow (i+m)_c$ 容量为 $+\infty$ ，费用为 f ；连边 $i_d \rightarrow (i+n)_c$ 容量为 $+\infty$ ，费用为 s 。
- 对于延迟送洗，连边 $i_d \rightarrow (i+1)_d$ 容量为 $+\infty$ ，费用为 0。

最后所有餐巾全部回收至 S 。

这是无源汇上下界最小费用可行流。在考虑流量下界之后， i_c 有 $-r_i$ 的净流量， i_d 有 r_i 的净流量。根据上下界网络流， $SS \rightarrow i_d$ 容量为 r_i ，实际含义为强制获得 r_i 条脏餐巾， $i_c \rightarrow TT$ 容量为 r_i ，实际含义为强制消耗 r_i 条干净的餐巾。

为了最小化费用， $i_c \rightarrow i_d$ 的流量一定恰为 r_i ，所以这条边可以省略掉。

注

上下界网络流的连边的实际含义给出了一个直接使用费用流的做法。这种做法连边较少一些，效率更高。其本质是注意到 S 的流量一定由 $SS \rightarrow i_d \rightarrow S$ 给出，这一段的费用均为 0，所以 S 的连边可以全部融进 SS 。

代码。注意 LOJ 和洛谷输入格式不一样。

LOJ6009 软件补丁

神秘题目，不建议做。状压 + SPFA 可以通过，不知道复杂度为什么是对的。

代码。

LOJ6010 数字梯形

通过拆点连容量为 1 的边限制每个点只能被选一次，边的容量设置为 1 限制每条边只能被选一次。

题目要求最大和，所以将所有费用取反。初始残量网络无环，可以求解。

代码。

LOJ6011 运输问题

对于最大费用，初始残量网络无环，可直接费用流。

代码。

LOJ6012 分配问题

二分图最大权完美匹配。费用流即可。

代码。

LOJ6013 负载均衡

类似上下界网络流，多的货物从源点送进来，少的货物是送到了汇点。

求出平均值 avg 。若 $a_i > avg$ 则连边 $S \rightarrow i$ 容量为 $a_i - avg$ ，费用为 0；否则连边 $i \rightarrow T$ 容量为 $avg - a_i$ ，费用为 0。相邻点之间连容量为 $+\infty$ ，费用为 1 的边。

代码。

*LOJ6014 最长 k 可重区间集

如果一组方案符合条件，那么一定能用不超过 k 条链把区间串起来，满足每条链的区间两两不交。相反，任何能用 k 条链串起来的区间集合均符合题目要求。因此考虑用 k 条链把区间串起来。

为限制每个区间只能贡献一次，拆点 L_i 和 R_i ，连边 $L_i \rightarrow R_i$ 容量为 1，费用为 $r_i - l_i$ 。此外，若 $r_i \leq l_j$ ，则连边 $R_i \rightarrow L_j$ 容量为 1，代价为 0。 $S \rightarrow L$ ， $R \rightarrow T$ 连边容量为 1，费用为 0。为限制流量不超过 k ，新建 T' ，连边 $T \rightarrow T'$ 容量为 k ，费用为 0。连边不成环，取反之后费用流即可。
 $m = \mathcal{O}(n^2)$ ，不是很优秀。

我们发现大部分的边都用于连接两个不相交的区间。转换一下思路，将每个坐标看成点，这样就只需在相邻两个点之间连边，即可描述所有连接两个不相交的区间的边。

具体地，将所有端点离散化，从小到大依次为 $x_1, x_2, \dots, x_t$ 。令 $x_0 = 0$ 。对所有 $0 \leq i < t$ ，连边 $i \rightarrow i + 1$ 容量为 k ，费用为 0。 $0 \rightarrow 1$ 的边限制了最大流不超过 k 。此外，对所有区间 $[l, r]$ ，令其

端点分别为 $x_i = l$ 和 $x_j = r$ ，则连边 $i \rightarrow j$ 容量为 1，费用为 $r - l$ 。 $m = \mathcal{O}(n)$ 。

代码。

LOJ6015 星际转移

一艘太空船停靠的站点随时间的变化而变化，启发我们使用 **分层图** 来刻画整个星际转移过程。

考虑从时刻 t 扩展到时刻 $t + 1$ 。

对每艘太空船 S 求出它在 t 时刻停靠的站点 $S(t)$ 和在 $t + 1$ 时刻停靠的站点 $S(t + 1)$ 。 $S(t)$ 上的至多 h_S 个人可以通过这艘太空船在时刻 t 到 $t + 1$ 之间移动到 $S(t + 1)$ （假定太空船每到达一个站点，就往站点处卸下所有的乘客）。故考虑从 t 时刻的 $S(t)$ 向 $t + 1$ 时刻的 $S(t + 1)$ 连边，容量为 h_S 。

此外，由于乘客可以在太站等待，且太空站容量无限，所以 t 时刻的 i 向 $t + 1$ 时刻的 i 连边，容量为 $+\infty$ 。

注意地球 0 和月亮 -1 较为特殊，不需要为它们新建节点（建完之后发现没有必要）。

从 1 到 $+\infty$ 枚举答案 t ，如果 t 时刻 $0 \rightarrow -1$ 的最大流不小于需要转移的人数 k ，则 t 即为所求。

使用并查集判断无解。

代码。

LOJ6121 孤岛营救

注意到钥匙种类很少，所以每种钥匙是否拥有的总状态数很少，状压 BFS 即可。时间复杂度 $\mathcal{O}(nm2^p)$ 。

代码。

LOJ6122 航空路线

题目相当于找到从 1 到 n 的两条只在 1 和 n 相交的路径，使得它们的长度之和最大。

一个点只能被经过一次，考虑拆点。1 和 n 的 in, out 之间连容量为 2 的边，剩下的连容量为 1 的边。若 i, j ($i < j$) 在原图有边，那么从 out_i 向 in_j 连容量为 1 的边（特判 $1 \rightarrow n \rightarrow 1$ 的情况）。有解当且仅当 in_1 到 out_n 的最大流等于 2。

为最大化路径长度之和，我们让每个点被流过时产生贡献 1，求最大费用。因为只会从编号小的点向编号大的点连边，所以网络无环，可直接取反后求最小费用。

数据没有 $n = 1$ 的情况。

代码。

本题也可以 DP：设两条路径当前端点分别在 i, j 时长度之和的最大值为 $f_{i,j}$ 。枚举 $k > \max(i, j)$ ，若 i, k 有边则可转移到 $f_{k,j}$ ；若 j, k 有边则可转移到 $f_{i,k}$ 。若 i, j 均与 n 直接相连则可用 $f_{i,j} + 1$ 更新答案。初始值 $f_{1,1} = 1$ 。为输出方案，需记录每个状态的决策点。时间复杂度 $\mathcal{O}(n^3)$ 。

LOJ6223 汽车加油行驶问题

设 $f_{i,j,k}$ 表示走到 (i, j) 且油箱剩余 k 单位的最小代价，Dijkstra 即可。放在网络流 24 题里很奇怪。

设 $N = n^2k$ ，时间复杂度 $\mathcal{O}(N \log N)$ 。代码。

LOJ6224 深海机器人

用建 **平行边** 的方式限制一条边的权值只贡献一次，即连边 $in \rightarrow out$ 容量为 1，费用为 w ；以及容量为 $+\infty$ ，费用为 0。建图方式显然，最大费用最大流即可。

代码。

LOJ6225 火星探险

首先拆点 in, out 点权转边权。

一个石块只能被采集一次，但每个位置可以被多次经过。若某个位置上有石块，则连边 $in \rightarrow out$ 容量为 1，费用为 1；以及容量为 $+\infty$ ，费用为 0。最大费用最大流即可。输出方案则记录每个位置有多少向右和向下的流量，从 $(1, 1)$ 开始 DFS。

代码。

LOJ6226 骑士共存

题图提示我们对网格图黑白染色，则有限制的两个格子之间颜色不同。二分图最大独立集即可。

代码。

LOJ6226 最长 k 可重线段集

平行于 y 轴的线段相当于闭区间 $[x, x]$ ，所以无法直接套用最长 k 可重区间集的做法。

稍作修补，拆点 in_x 和 out_x 。 (l, r) 连边 $out_l \rightarrow in_r$ ， $[x, x]$ 连边 $in_x \rightarrow out_x$ 。此外，连边 $in_x \rightarrow out_x$ 容量为 k ，代价为 0。

拆点还是有些麻烦，直接令 $in_x = 2x$ ， $out_x = 2x + 1$ 即可转化为最长 k 可重区间集。

代码。

7.2 简单题

P2936 [USACO09JAN] Total Flow S

最大流模板题。

P1345 [USACO5.4] 奶牛的电信 Telecowmunication

有向图点边转化基础练习题。

P2057 [SHOI2007] 善意的投票 / [JLOI2010] 冠军调查

集合划分模型，分析见 6.2 小节。

代码。

P2153 [SDOI2009] 晨跑

将 $2 \sim n - 1$ 拆点后跑最小费用最大流即可。

代码。

P2604 [ZJOI2010] 网络扩容

第一问建容量为 w 费用为 0 的边跑最大流，第二问新建容量为 $+\infty$ 费用为 c 的边跑限制流量 k 的费用流。

代码。

P1231 教辅的组成

模型显然，将中间点拆点连容量为 1 的边限制度数，跑最大流即可。

代码。

P3410 拍照

最大权闭合子图板子题。

CF1082G Petya and Graph

如果一条边被选，则其两个端点必须选。最大权闭合子图模型。

代码。

P5192【模板】有源汇上下界最大流

对每一天 i ，从 S 向 i 连容量 $[0, D_i]$ 的边，从 i 向每个当天可拍的少女 j 连流量限制为 $[L_j, R_j]$ 的边。

对每个少女 j ，向 T 连流量限制为 $[G_j, \infty]$ 的边，跑有源汇上下界最大流。

代码。

P4553 80 人环游世界

有源汇上下界费用流模板题。

代码。

P2045 方格取数加强版

有向图点边转化：将每个点 i 拆成 i_{in} 和 i_{out} 。

建平行边表示每个点的贡献只会算一次： $i_{in} \rightarrow i_{out}$ 连容量 1，边权 c_i 的边；再连一条容量为 $k - 1$ ，边权为 0 的边。

每个网格的出点向右侧和下侧的网格的入点连边。 $(1, 1)_{in} \rightarrow (n, n)_{out}$ 的最小费用最大流即为所求。

代码。

P4843 清理雪道

有源汇上下界最小流模板题。

从 $S \rightarrow i \rightarrow T$ 连容量为 $+\infty$ 的边。原图的边的流量限制为 $[1, +\infty]$ 表示这条边必须被流。代码。

如果点边转化为 DAG 最小可交路径覆盖，则时间复杂度 $\mathcal{O}(m^3 + \text{MaxFlow}(m, m^2))$ 。因为 $m = \mathcal{O}(n^2)$ 所以无法接受。

*P2805 [NOI2009] 植物大战僵尸

对于每个植物，从它的攻击位置向它连边，表示若选择攻击位置则必须选择该植物。

因为环以及能到达环的点都不可以选择，所以对反图拓扑排序。对遍历到的节点求解最大权闭合子图问题。

代码。

P4313 文理分科

考虑文理分科模型。

令学生与 S 相连表示选理科，与 T 相连表示选文科。对所有贡献求和，最小化减去的代价。据此， $S \rightarrow (i, j)$ 容量为 $science_{i,j}$ ， $(i, j) \rightarrow T$ 容量为 $art_{i,j}$ 。

对于联合贡献，新建点 $u_{i,j} \rightarrow (x, y)$ 容量为 $+\infty$ ，其中 (x, y) 为 (i, j) 及与其相邻的总共五个格子， $S \rightarrow u_{i,j}$ 容量为 $same_science_{i,j}$ ；新建点 $(x, y) \rightarrow v_{i,j}$ 容量为 $+\infty$ ， $v_{i,j} \rightarrow T$ 容量为 $same_art_{i,j}$ 。

代码。

P1361 小 M 的作物

考虑文理分科模型。

令作物与 S 相连表示种在 A 地，与 T 相连表示种在 B 地。因集合划分模型只能处理最小化代价，故先将所有贡献加入，尝试最小化扣除贡献。据此， $S \rightarrow i$ 容量 a_i ， $i \rightarrow T$ 容量 b_i 。

对于联合贡献，同样先将贡献 $c_1 + c_2$ 加入并最小化扣除贡献。易知 $S \rightarrow c_A$ 容量为 c_1 ， $c_A \rightarrow I$ 容量为 $+\infty$ ， $I \rightarrow c_B$ 容量为 $+\infty$ ， $c_B \rightarrow T$ 容量 c_2 ，其中 I 为涉及的作物集合。

代码。

CHANGE LOG

- 2021.12.5：更换模板代码。
- 2022.1.11：重构文章，新增网络流的应用与模型。
- 2022.1.13：新增上下界网络流部分。
- 2022.5.11：重构文章，更换模板代码。
- 2025.6.10：重构文章，新增切糕模型、集合选择模型和最大密度子图。

参考资料

- [网络流与二分图 学习笔记](#) —— chenxia25。
- [NOI 一轮复习 I: 二分图网络流](#) —— ix35。

第二章：

- [最大流](#) —— OI Wiki。

第三章

- [费用流](#) —— OI Wiki。

第六章

- [P3227 建图证明](#) —— dengyaotriangle。